

■ STRUCTURED INTELLIGENCE • TECHNICAL WALKTHROUGH

Wie Doctus technisch funktioniert.

Eine gründliche Tour durch Komponenten, Datenflüsse und Implementierungsprinzipien — vom Source Connector über COBOL-Analyse und RAG bis zum Multi-Panel-Frontend.

Zielgruppe: Entwicklerinnen und Entwickler, die Doctus erweitern, betreiben oder in Kundenumgebungen integrieren.

PRODUKT

Self-hosted Legacy Code Intelligence

KERN

Code, Dokumente und Beziehungen gemeinsam nutzbar machen

LEITPRINZIP

Quelle und Kontext bleiben nachvollziehbar

■ EINFACH ERKLÄRT • THEMA 01

Worum es bei Doctus geht.

Doctus hilft dabei, große Mengen an altem Code und begleitenden Unterlagen wieder verständlich zu machen. Statt sich durch viele Dateien und Ordner zu arbeiten, können Teams an einem Ort sehen, was vorhanden ist und wie Dinge zusammenhängen.

Das Werkzeug ersetzt die Originale nicht. Es macht sie auffindbar und verknüpft sie so, dass man bei einer Frage immer wieder zur passenden Datei, Seite oder Zeile zurückkehren kann.

Merksatz: Doctus schafft einen gemeinsamen Arbeitsplatz für gewachsenes Wissen.

Doctus übersetzt gewachsene Informationen in navigierbares Wissen.

Das System hält drei Arten von Informationen zusammen: originale Quellen, daraus extrahierte Struktur und semantischen Kontext. Jede Nutzerfunktion kombiniert mindestens zwei davon.

ORIGINAL	STRUKTUR	BEDEUTUNG
Quellen COBOL-Dateien, Copybooks, Git-Historie sowie begleitende Dokumente aus Confluence, Jira, WebDAV, Ordnern und Uploads.	Entitäten & Kanten Programme, Sections, Paragraphs, Datenfelder und SQL-Blöcke sowie Aufrufe, Kopien und Verwendungen.	Chunks & Vektoren Durchsuchbare Textabschnitte mit Metadaten und BGE-M3-Embeddings für semantisches Retrieval.

Arbeitsprinzip: Antworten und Visualisierungen sollen nicht vom Originalbestand entkoppelt sein. Dateien, Seiten und physische Zeilen bilden deshalb wiederverwendbare Anker.

Drei Blickwinkel auf dieselben Informationen.

Eine Datei ist zunächst einfach ein Original: etwa ein COBOL-Programm, ein Copybook oder eine Beschreibung aus Confluence. Doctus bewahrt diese Originale, damit ihre Aussage jederzeit überprüfbar bleibt.

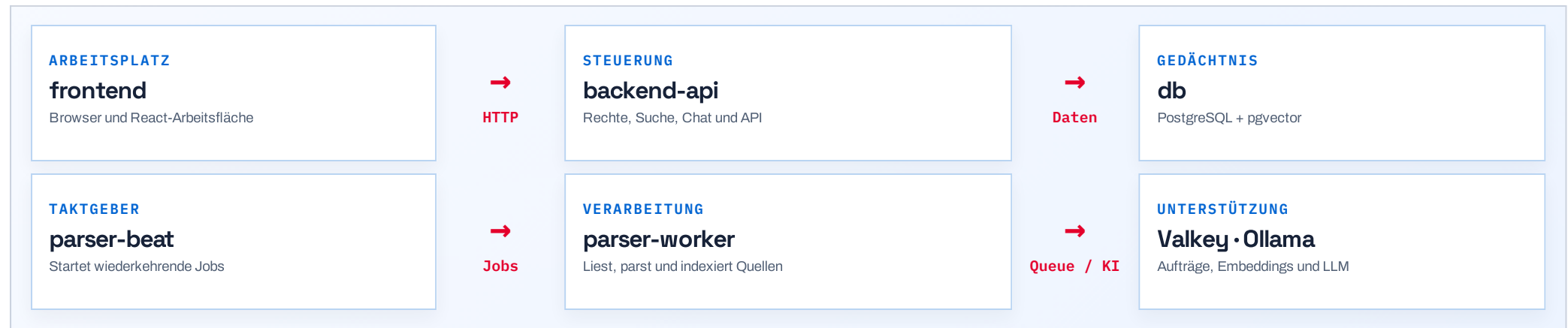
Zusätzlich erkennt das System Bausteine und Verbindungen im Code und macht Texte nach ihrer Bedeutung durchsuchbar. So kann man sowohl gezielt nach einem Feld suchen als auch Fragen stellen, deren Antwort über mehrere Unterlagen verteilt ist.

Merksatz: Original, Struktur und Bedeutung ergänzen sich – keiner dieser Blickwinkel genügt allein.

■ SYSTEM ARCHITECTURE

Sieben Compose-Services bilden die lokale Laufzeit.

Frontend, API, Hintergrundverarbeitung, Datenhaltung, Queue und KI sind getrennt deploybar, laufen aber in einem gemeinsamen Compose-Netz.



Compose-Services: `frontend`, `backend-api`, `parser-worker`, `parser-beat`, `db`, `redis` (Valkey) und `ollama`.

■ EINFACH ERKLÄRT • THEMA 03

Die Architektur als kleine Werkstatt.

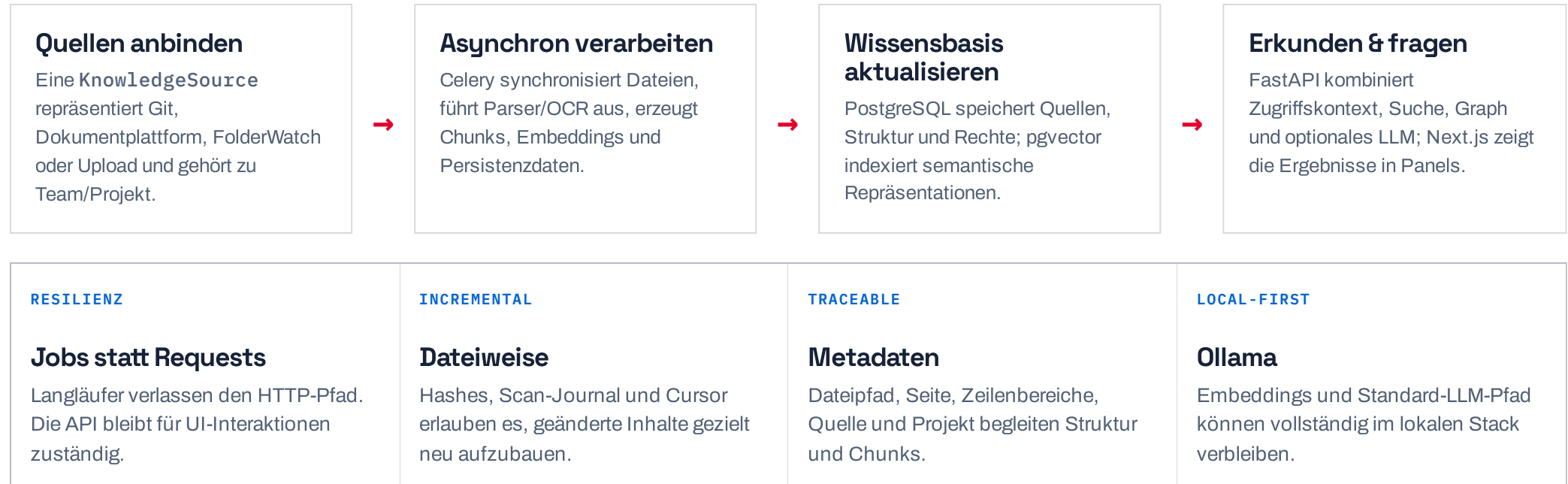
Im Browser arbeitet die Person mit Doctus. Die API ist die Empfangstheke: Sie prüft Rechte, nimmt Anfragen an und liefert Ergebnisse zurück. Für große oder langsame Aufgaben gibt es eigene Hintergrundhelfer, damit niemand warten muss.

Datenbank, Warteschlange und KI-Modelle sind die gemeinsame Infrastruktur im Hintergrund. Sie speichern Wissen, verteilen Arbeit und helfen bei Suche oder Textverstehen. Die Abbildung auf der vorherigen Folie zeigt, wer mit wem spricht.

Merksatz: Jeder Teil hat eine klare Aufgabe; zusammen wirken sie wie ein eingespieltes Team.

■ END-TO-END FLOW

Vom Eingang einer Quelle bis zur Arbeit im Browser.



Was nach dem Anschließen einer Quelle passiert.

Eine Quelle kann ein Git-Repository, ein Ordner oder eine Dokumentplattform sein. Doctus holt die Inhalte ab und gibt die aufwendige Arbeit an Hintergrundprozesse weiter. Das geschieht nicht mitten in einer Browseranfrage.

Danach liegen die Informationen geordnet in der Wissensbasis. Erst dann zeigt der Browser die Dateien, Beziehungen und Suchergebnisse an. Dadurch bleibt die Oberfläche schnell, auch wenn im Hintergrund viele Dokumente verarbeitet werden.

Merksatz: Erst sammeln und aufbereiten, dann bequem im Browser damit arbeiten.

■ SOURCE CONNECTORS

Wissensquellen werden über ein gemeinsames Ingestion-Modell angebunden.

QUELLE	VERARBEITUNGSPRINZIP	ERGEBNIS IM SYSTEM
Git	Bare Mirror und branch-isolierte Worktrees; Delta-Synchronisierung und dateiweise Verarbeitung.	Repositorydateien, COBOL-Entitäten/Kanten, Code-Chunks und Embeddings.
Confluence / Jira	Connector lädt fachliche Seiten bzw. Vorgänge und normalisiert sie als Dokumentinhalt.	Dokument-Chunks mit Quelle, Titel und Metadaten für Suche und Chat.
WebDAV / FolderWatch	Dateien werden heruntergeladen bzw. read-only aus /watched gelesen; Scanintervall steuert Auto-Sync.	Text-/OCR-Ergebnis, Chunks, Embeddings und Fortschrittsstatus.
Lokaler Upload	Datei wird im Uploadpfad persistiert und durch die Dokumentpipeline verarbeitet.	Projektgebundene Knowledge Source und referenzierbare Inhalte.

Gemeinsame Abstraktion: KnowledgeSource trägt Verbindungsdaten, Sync-Status, Fortschritt, Branch, Cursor und fachliche Kontextnotiz. Dadurch verhalten sich Quellen im Workspace einheitlich.

■ EINFACH ERKLÄRT • THEMA 05

Verschiedene Ablagen, ein gemeinsamer Eingang.

Teams legen Wissen an sehr unterschiedlichen Orten ab: im Quellcode, in Wiki-Seiten, in Tickets oder in gemeinsamen Ordnern. Doctus behandelt all diese Orte als Quellen, auch wenn die Technik dahinter verschieden ist.

Für jede Quelle merkt sich das System unter anderem ihren Zustand und ihren letzten Abgleich. Deshalb sieht die Arbeit mit einer hochgeladenen Datei im Arbeitsbereich ähnlich aus wie die Arbeit mit einem ganzen Repository.

Merksatz: Die Herkunft bleibt sichtbar, die Bedienung wird trotzdem einheitlich.

Celery organisiert jede zeitintensive Verarbeitung als nachvollziehbaren Job.

Warum Worker?

Clone, OCR, Parsing und Modellaufrufe können lange dauern. Sie dürfen weder Browser-Requests blockieren noch den API-Prozess in eine instabile Rechenrolle drücken.

01 Broker

Valkey ist Redis-kompatibel und übergibt Aufgaben an den Parser-Worker.

02 Scheduler

Celery Beat startet periodische Aufgaben, etwa FolderWatch-Scans.

03 Jobstatus

Quellen und Jobs speichern Fortschritt, Meldungen und Zeitinformationen für UI und Diagnostik.

Verarbeitungsfolge pro Datei

01 Inhalt einlesen

Dateityp bestimmen, Text extrahieren; gescannte PDF können über Tesseract OCR laufen.

02 Struktur / Text zerlegen

COBOL nutzt den spezialisierten Parser; andere Inhalte verwenden generisches Chunking.

03 Embedding erzeugen

Chunks werden über Ollama mit `bge-m3` vektorisiert; Sub-Batches steuern den Modellaufruf.

04 Transaktional speichern

Chunks, Entitäten und Kanten werden aktualisiert; veränderte Chunks behalten nachvollziehbare Links.

■ EINFACH ERKLÄRT • THEMA 06

Warum schwere Arbeit im Hintergrund läuft.

Das Lesen vieler Dateien, das Erkennen von Code und das Umwandeln von gescannten Seiten kann dauern. Würde das direkt beim Klick im Browser passieren, sähe die Anwendung schnell aus, als wäre sie eingefroren.

Darum erstellt Doctus dafür Arbeitsaufträge. Hintergrundhelfer erledigen sie nacheinander oder parallel und melden Fortschritt und Probleme zurück. In der Oberfläche kann man dabei weiterhin normal arbeiten.

Merksatz: Lange Aufgaben werden geplant und überwacht, statt den Arbeitsplatz zu blockieren.

Die Codeanalyse baut eine deterministische Strukturkarte des Bestands.

Der Parser arbeitet lokal und regelbasiert. Er interpretiert den Quelltext, ohne ein LLM zum Parsen einzusetzen, und bewahrt physische Zeilen als stabile Navigationseinheit.



Zentrale Invariante: Copybooks werden nicht in den Programmtext expandiert. Sie bleiben eigene Quellen; die Nutzung wird über COPY-Kanten und vererbte Feldindizes modelliert.

Wie aus altem COBOL eine lesbare Landkarte wird.

COBOL-Code ist nicht nur Text. Er enthält Programme, Abschnitte, Datenfelder und Befehle, die aufeinander verweisen. Doctus liest diese Regeln mit einem festen Parser aus, also mit einem Werkzeug, das nach klaren Regeln arbeitet.

Dadurch kann das System sagen, welche Teile zusammengehören und wo sie im Original stehen. Das ist zuverlässiger als freies Raten und hilft besonders dann, wenn später jemand eine bestimmte Zeile oder ein Datenfeld prüfen möchte.

Merksatz: Der Parser baut eine überprüfbare Karte des Codes – Zeile für Zeile.

■ **PARSER OUTPUT**

Entitäten und Kanten bilden den technischen Code Graph.

ENTITY TYPES	HIERARCHY	IDENTITY	SOURCE ANCHOR
CodeEntity program, copybook, section, paragraph, data_item, file_fd und sql_block.	parent_id Programm → Section → Paragraph sowie Programm/Copybook → Datenfelder werden als Hierarchie im selben Entity-Modell gespeichert.	qualified_name Ein qualifizierter Name wie PROGRAM.SECTION.PARAGRAPH stabilisiert Deep-Links und Upserts.	Datei & Zeile Jede Entität trägt Datei sowie Start-/Endzeile; Metadaten halten z. B. PIC, Level, OCCURS oder SQL-Typ.

KANTENART	BEISPIEL	AUFLÖSUNGSBEREICH
CALL / COPY	Programm ruft anderes Programm auf; Programm bindet Copybook ein.	Global innerhalb einer Quelle; Ziel kann erst im Nachlauf vorhanden sein.
PERFORM / GOTO	Kontrollfluss zwischen Paragraphen/Sections.	Lokal im Programm, damit gleichnamige Ziele nicht falsch verknüpft werden.
USES / DEFINES	Paragraph oder SQL-Block verwendet Datenfeld.	Lokal bzw. mit Copybook-Feldvererbung.
READS / WRITES	Datei-/Datensatzoperationen.	Als semantische Beziehung für Analyse und Graphen.

■ EINFACH ERKLÄRT • THEMA 08

Knoten und Pfeile machen Beziehungen sichtbar.

Doctus speichert wichtige Dinge im Code als einzelne Objekte, zum Beispiel Programme, Abschnitte oder Datenfelder. Jedes dieser Objekte behält seinen Namen sowie den Ort in der Datei.

Zwischen den Objekten speichert das System Pfeile: Ein Programm ruft ein anderes auf, ein Abschnitt benutzt ein Feld oder eine Datei wird gelesen. Daraus entsteht ein Netz, in dem man Ursachen und Folgen leichter verfolgen kann.

Merksatz: Objekte beantworten „was ist das?“, Beziehungen beantworten „was hängt daran?“.

■ PERSISTENCE & RESOLUTION

Der Parser ist DB-frei; Persistenz und Kantenauflösung sind eigene Prozessschritte.

Pass 0 • Index

Der Git-Prozess erstellt einen Copybook-Index über den Bestand, damit COPY und geerbte Felder adressierbar werden.

**Pass 1 • Persist**

`persist_parse_result()` schreibt Entity- und Kantenresultate pro Datei per Upsert.

**Pass 2 • Resolve**

Globale Kanten bekommen ihren konkreten Zielknoten, sobald der Zielbestand im Index vorliegt.

**Graph verfügbar**

Call Graph, Referenzen, Suche und graph-erweitertes Retrieval können auf Kanten zugreifen.

Warum Upsert statt Löschen? Ein Reparse einer Datei darf keine eingehenden Kanten aus unveränderten Dateien kaskadierend entfernen. Nur verschwundene Entities werden gezielt gelöscht; Kanten der gerade verarbeiteten Quelle werden ersetzt.

■ EINFACH ERKLÄRT • THEMA 09

Erst sammeln, dann sauber verbinden.

Beim Einlesen einer Datei wird nicht alles auf einmal gelöst. Zuerst speichert Doctus, was es in dieser Datei sicher erkannt hat. Danach kann es Verbindungen zu anderen Dateien herstellen, sobald die nötigen Gegenstücke bekannt sind.

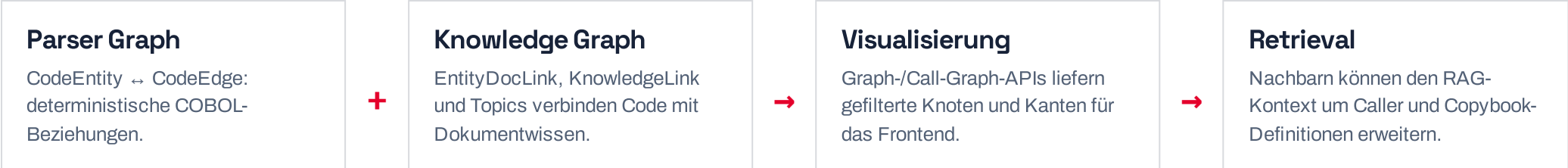
Das ist wichtig, weil ein Programm auf etwas verweisen kann, das erst später verarbeitet wird. Beim erneuten Einlesen wird nur das geändert, was wirklich betroffen ist; bereits bekannte Verbindungen aus anderen Dateien bleiben erhalten.

Merksatz: Die Verarbeitung ist schrittweise, damit große Bestände stabil wachsen können.

Der Knowledge Graph ist kein separater Dienst, sondern eine Projektion der relationalen Wissensbasis.

Graphansichten und Call Graphs werden aus `CodeEntity`, `CodeEdge`, Dokumentlinks, Topics und Projekt-/Quellengrenzen aufgebaut. PostgreSQL bleibt dabei System of Record.

NODES	EDGES	TRAVERSAL
Technische und fachliche Objekte Codeentitäten, Dokumente, Quellen, Projekte und Topics können als Knoten in unterschiedlichen Ansichten auftreten.	Explizite Beziehungen Parserkanten beschreiben Codeabhängigkeiten. Entity-Document- und Knowledge-Links ergänzen fachliche Zusammenhänge.	Kontextabhängige Sicht Call Graphs nutzen 1–3 Hops und Filter. Sichtbarkeit wird gegen Projekt- und Teamzugriff geprüft.



Ein Graph ist eine Karte für Zusammenhänge.

Man kann sich den Knowledge Graph wie eine Stadtkarte vorstellen. Gebäude sind Programme, Dokumente oder Themen. Straßen zeigen, welche Dinge zusammengehören oder sich gegenseitig benutzen.

Die Karte wird nicht als zweites, getrenntes System geführt. Doctus baut sie aus den bereits gespeicherten Daten und zeigt immer nur den Ausschnitt, den eine Person sehen darf und gerade braucht.

Merksatz: Der Graph macht vorhandene Daten anschaulich, ohne eine neue Wahrheit daneben zu erfinden.

Dokumente werden zu metadatenreichen Chunks — nicht zu einem unstrukturierten Textblock.

Extraktion

01 Text first

PDFs mit Textlayer und Office-/Textformate werden nativ gelesen.

02 OCR fallback

Bildbasierte PDFs können Seiten rasterisieren und Tesseract OCR für Deutsch/Englisch verwenden.

03 Inhalt schützen

Chunk-Inhalte werden in der Datenbank als verschlüsselte Werte gespeichert.

Chunk-Metadaten

01 Position

`file_path`, Start-/Endzeile und bei Dokumenten Seitenzahl erlauben die Rückkehr zur Fundstelle.

02 Herkunft

`source_id`, `project_id`, Titel und Quelltyp halten Kontext und Berechtigungsbezug fest.

03 Semantik

Ein 1024-dimensionalen Vektor repräsentiert den Inhalt für Ähnlichkeitssuche über `pgvector`.

Dokumente werden in gut auffindbare Abschnitte geteilt.

Ein langes PDF oder Wiki-Dokument ist schwer als Ganzes zu durchsuchen. Deshalb teilt Doctus den Inhalt in kleinere, sinnvolle Abschnitte. Zu jedem Abschnitt merkt sich das System, aus welcher Datei und möglichst von welcher Seite er stammt.

Wenn ein PDF nur aus Bildern besteht, kann es zuerst per Texterkennung lesbar gemacht werden. Der Inhalt wird geschützt gespeichert und zusätzlich so vorbereitet, dass ähnliche Aussagen auch dann gefunden werden, wenn andere Worte verwendet wurden.

Merksatz: Kleine Abschnitte mit Herkunft machen Dokumentwissen suchbar und belegbar.

■ DATABASE OVERVIEW

PostgreSQL + pgvector ist der gemeinsame Speicher für Betriebs- und Wissensdaten.

BEREICH	ZENTRALE TABELLEN	AUFGABE
Identität & Zugriff	users, teams, team_memberships, projects, project_memberships	Lokale Identitäten, Organisationseinheiten und projektfeine Rechte.
Quellen & Synchronisierung	knowledge_sources, source_scan_files	Connector-Konfiguration, Status, Cursor, Datei-Journal und Fehler-/Fallbackinformation.
Struktur & Retrieval	code_entities, code_edges, document_chunks	Codehierarchie, Beziehungen, Textsegmente und Vektorindex.
Arbeitswissen	entity_doc_links, knowledge_links, topics, topic_nodes	Kuratiert bzw. automatisch erzeugte fachliche Querverbindungen.
Interaktion & Betrieb	chat_sessions, chat_messages, Job-/Diagnosemodelle	Unterhaltungen, Workspace-Snapshots, Feedback und Nachvollziehbarkeit.

Indexierung: `document_chunks.embedding` verwendet einen HNSW-Cosine-Index (`m=16`, `ef_construction=64`). Daneben existieren B-Tree-Indizes für häufige Source-, Project-, File- und Entity-Zugriffe.

■ EINFACH ERKLÄRT • THEMA 12

Die Datenbank ist das Gedächtnis des Systems.

In der Datenbank liegen nicht nur Benutzer und Projekte. Sie enthält auch Quellen, den Verarbeitungsstand, erkannte Codebausteine, Dokumentabschnitte, Gespräche und die Verbindungen dazwischen.

Der Vektorindex ist ein besonderer Teil dieses Gedächtnisses: Er hilft dabei, Texte nach ihrer Bedeutung zu finden. Trotzdem bleibt die Datenbank die zentrale, überprüfbare Ablage für alle Informationen.

Merksatz: Ein gemeinsamer Speicher hält Betrieb, Rechte und Wissen konsistent zusammen.

Nutzerverwaltung beginnt lokal und wird durch Teams und Projekte verfeinert.

Der Einstieg ist eine lokale Nutzerverwaltung: beim ersten Start wird ein Bootstrap-Superuser erzeugt. Danach verwaltet die Anwendung Nutzer, Teams, Teammitgliedschaften und Projektmitgliedschaften.

USER	TEAM	PROJECT	MEMBERSHIP
Lokale Identität Username, optionale E-Mail/Name, Argon2id-Hash, Aktivstatus, Passwortwechsel-Flag und Loginstatus.	Organisationsgrenze Ein Team gruppiert Nutzer und ist die grundlegende Sichtbarkeitsgrenze für Quellen und Projekte.	Arbeitskontext Projekt ist oberster fachlicher Container. Quellen, Chunks, Entitäten und Kanten werden ihm zugeordnet.	Feingranularer Zugriff Projektmitgliedschaft bestimmt, wer innerhalb eines Teams ein konkretes Projekt verwenden darf.

Admin: Ein globaler Administrator verwaltet Teams/Nutzer. Die Erkennung erfolgt über das konfigurierte Admin-Konzept; Team- und Projektmitgliedschaften bleiben davon getrennte Domänenobjekte.

■ EINFACH ERKLÄRT • THEMA 13

Wer etwas sehen darf, wird in Stufen geregelt.

Zuerst gibt es einzelne Nutzerkonten. Sie können zu Teams gehören, und Teams können mehrere Projekte haben. So lässt sich Wissen nach Organisation und Arbeitsauftrag trennen.

Innerhalb eines Teams kann ein Projekt nochmals eigene Mitglieder haben. Eine Person sieht damit nicht automatisch alles, nur weil sie im selben Unternehmen oder Team arbeitet. Administratoren verwalten die Struktur, ohne die fachlichen Grenzen aufzuheben.

Merksatz: Zugriff wird vom Groben zum Feinen geregelt: Nutzer, Team, Projekt.

 AUTHENTICATION

Login erzeugt eine signierte, HTTP-only Anwendungssession.

01 • CREDENTIALS Login Request POST /auth/login prüft Benutzername und Passwort gegen die lokale <code>users</code>-Tabelle.	02 • PASSWORTPRÜFUNG Argon2id Der Hash wird verifiziert; bei geänderten Parametern kann ein erfolgreicher Login transparent rehashen.	03 • THROTTLE Redis + DB Fehlversuche fließen in Redis- und DB-Zähler; Sperrzeit folgt der höheren wirksamen Sperre.	04 • SESSION Cookie <code>doctus_session</code> ist signiert, HTTP-only, SameSite=Lax und 14 Tage gültig; Secure bei HTTPS.
--	---	--	---

BAUSTEIN	VERANTWORTUNG
<code>api/auth.py</code>	Login, Logout, Passwortwechsel, aktueller Nutzer und Login-Flow.
<code>core/passwords.py</code>	Hashing, Verifikation und Rehash-Entscheidung.
<code>core/login_throttle.py</code>	IP-/Username-bezogene Zähler und Sperrlogik.
<code>core/auth_dependency.py</code>	Cookie signieren, laden, maximale Sessiondauer prüfen und <code>get_current_user</code> bereitstellen.

■ EINFACH ERKLÄRT • THEMA 14

Ein Login hinterlässt einen geschützten Ausweis.

Beim Anmelden vergleicht Doctus die eingegebenen Zugangsdaten mit sicher gespeicherten Informationen. Das Passwort selbst wird nicht als lesbarer Text abgelegt, sondern nur in einer Form, die sich prüfen lässt.

Nach erfolgreichem Login erhält der Browser einen geschützten Sitzungsnachweis. Dieser Nachweis zeigt der Anwendung bei späteren Anfragen, wer angemeldet ist. Zu viele Fehlversuche werden gebremst, damit Rateversuche schwerer werden.

Merksatz: Der Browser bekommt einen geschützten Ausweis, nicht das Passwort.

■ AUTHORIZATION

Autorisierung kombiniert globale Adminrechte mit Team- und Projektzugriff.

EBENE	FRAGE	MECHANISMUS
Authentifiziert?	Darf die Anfrage grundsätzlich an geschützte Router?	FastAPI-Router verlangen <code>get_current_user</code> ; auth/system bleiben bewusst separat.
Administrator?	Darf der Nutzer organisationsweit verwalten oder Grenzen überblicken?	<code>require_admin</code> schützt z. B. Teams, Nutzer, Topics und Diagnosefunktionen.
Team sichtbar?	Gehört Projekt/Quelle zu einem Team, das der Nutzer sehen darf?	Gemeinsame Teamhelfer filtern Listen und prüfen Einzelressourcen; unsichtbare Ressourcen verhalten sich wie nicht gefunden.
Projektmitglied?	Darf ein Teammitglied dieses konkrete Projekt nutzen?	ProjectMembership ergänzt Team-Sichtbarkeit; Projektadmins verwalten Mitglieder und Access Requests.
Spezialrolle?	Darf ein Nutzer fachlich sensible Aktionen ausführen?	Die Rolle <code>pruefingenieur</code> ist für Compliance-Freigabe vorgesehen; <code>admin</code> verwaltet Mitgliedschaften.

Kontextsichtbarkeit: Codeanalyse kann pro Projekt explizit für den allgemeinen Kontext freigegeben werden. Standardmäßig bleibt Codeanalyse außerhalb des eigenen Projektkontexts verborgen.

■ EINFACH ERKLÄRT • THEMA 15

Anmelden ist nicht dasselbe wie berechtigt sein.

Ein Login zeigt nur, wer eine Person ist. Danach prüft Doctus bei jeder wichtigen Aktion zusätzlich, ob diese Person das gewünschte Team, Projekt oder sensible Funktion verwenden darf.

Das verhindert zum Beispiel, dass eine angemeldete Person allein durch das Raten einer Adresse auf ein fremdes Projekt zugreifen kann. Nicht sichtbare Inhalte verhalten sich für sie so, als gäbe es sie nicht.

Merksatz: Identität beantwortet „wer bist du?“, Berechtigung beantwortet „darfst du das?“.

Gespeicherte Inhalte und Connector-Geheimnisse bleiben verschlüsselt im lokalen Datenbestand.

FERNET AT REST

EncryptedString

Tokens, Dokument-Chunks, Chat-Nachrichten und relevante Compliance-Inhalte verwenden einen Fernet-basierten Verschlüsselungstyp.

KEY SEPARATION

Zwei Secrets

MASTER_ENCRYPTION_KEY verschlüsselt Daten.
SESSION_SECRET_KEY signiert
Anwendungssessions.

NETWORK EXPOSURE

Local services

PostgreSQL, Valkey und Ollama werden im Compose-Standard nur auf dem Host-Loopback veröffentlicht; Container sprechen intern über Service-Namen.

Entwicklungsfolge: Verschlüsselte Textspalten können nicht einfach per SQL-ILIKE durchsucht werden. Pfade, Metadaten und Vektoren bleiben deshalb wichtige Retrieval-/Filtermechanismen; begrenzte exakte Textprüfungen erfolgen nach dem Laden in Python.

■ EINFACH ERKLÄRT • THEMA 16

Schutz für Inhalte und Schlüssel.

Sensible Inhalte und Zugangsdaten von Quellen werden verschlüsselt gespeichert. Selbst ein Blick direkt in die Datenbank soll daraus nicht ohne den passenden Schlüssel lesbare Informationen machen.

Die Schlüssel für gespeicherte Daten und für Browser-Sitzungen haben verschiedene Aufgaben. Außerdem bleiben interne Dienste nach Möglichkeit im lokalen Netz. Das verringert die Zahl der Stellen, an denen vertrauliche Daten sichtbar werden können.

Merksatz: Schutz entsteht aus mehreren Schichten: Verschlüsselung, getrennten Schlüsseln und kleinen Netzgrenzen.

RAG recherchiert, prüft und baut das belegte Kontextpaket.

RAG ist die Recherche- und Kontextschicht, nicht die Antwortgenerierung: Sie legt fest, welche zugänglichen Quellen zur Frage passen, und übergibt dem LLM daraus ein begrenztes, referenzierbares Arbeitsmaterial.



VECTOR	EXACT	SCOPED
Semantisch pgvector sortiert Chunks nach Cosine-Distanz zum Query-Embedding.	Referenzbewusst Seitenzahlen und Abschnittsnummern sind semantisch schwach, aber fachlich präzise; sie werden zusätzlich behandelt.	Zugriff zuerst Die Basisabfrage ist bereits auf Projekt/Quelle und Berechtigung eingegrenzt, bevor Retrieval-Ergebnisse entstehen.

■ EINFACH ERKLÄRT • THEMA 17

RAG ist die Recherche vor der Antwort.

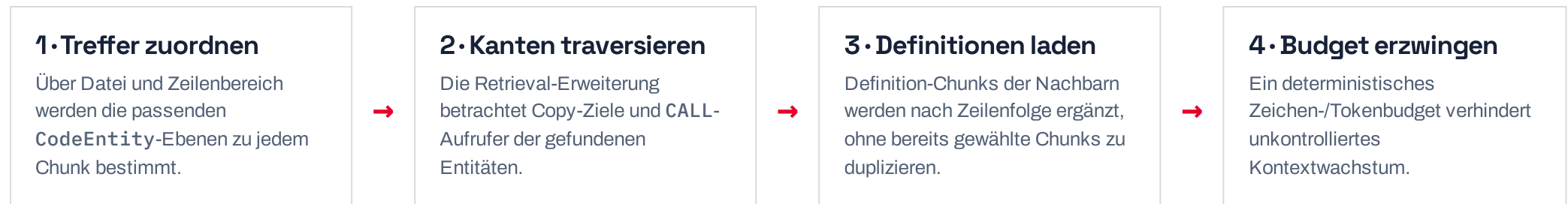
Wenn jemand eine Frage stellt, sucht RAG zuerst nach passenden Informationen. Es berücksichtigt dabei, was die Person sehen darf, welche Quelle gemeint ist und welche Textstellen zur Frage passen.

Das Ergebnis ist noch keine formulierte Antwort. RAG stellt ein kleines Paket aus passenden Ausschnitten, ihrer Herkunft und möglichen Belegen zusammen. Erst dieses Paket bekommt das Sprachmodell als Arbeitsmaterial.

Merksatz: RAG entscheidet, welches belegte Material zur Frage auf den Tisch kommt.

Der Graph ergänzt Vektortreffer um technisch benachbarten Kontext.

Vektorähnlichkeit findet den passenden Ausschnitt. Für Codefragen reicht dieser Ausschnitt oft nicht: Ein Treffer in einem Programm benötigt zum Verständnis möglicherweise das verwendete Copybook oder die Aufruferkette.



Resultat: Die Antwort kann den semantisch ähnlichen Text mit dem strukturell notwendigen technischen Umfeld kombinieren. Eine Sichtbarkeitsprüfung filtert Graphnachbarn erneut im Kontext der Anfrage.

Manchmal gehört mehr Kontext dazu.

Ein einzelner Treffer im Code kann unverständlich sein, wenn er auf ein anderes Programm oder ein Copybook verweist. Deshalb kann Doctus über bekannte Verbindungen noch nahe liegende, wichtige Stellen hinzunehmen.

Dabei gibt es Grenzen: Es wird nur innerhalb der erlaubten Sichtbarkeit gearbeitet und nur so viel Kontext ergänzt, wie in das vorgesehene Budget passt. So wird aus einem Zufallstreffer keine unübersichtliche Materialsammlung.

Merksatz: Der Graph ergänzt Nachbarschaft, RAG behält Relevanz und Menge im Blick.

LLM: begründen und erklären. RAG: Belege liefern.

Es liest RAG-Treffer nicht bloß vor: Das LLM verbindet die ausgewählten Ausschnitte, erklärt Zusammenhänge, zieht kenntlich gemachte Schlussfolgerungen und strukturiert die Antwort zur konkreten Frage. RAG bleibt dabei der kontrollierte Researchweg.

AUFGABE	LLM-ROLLE	GRENZE ZU RAG / SYSTEM
Einordnen & erklären	Verdichtet mehrere Belege, erläutert Ursache-Wirkung und passt Tiefe, Sprache und Aufbau an die Frage an.	RAG wählt und begrenzt die zugänglichen Belege; das LLM erweitert sie nicht eigenmächtig zur Wissensbasis.
Schlussfolgern	Leitet aus mehreren Ausschnitten eine nachvollziehbare Antwort ab und kennzeichnet Unsicherheit oder fehlende Information.	Keine Primärquelle erfinden oder Vermutung als belegte Tatsache ausgeben.
Agentischer Codekontext	Bei Repo-Kontext kann der Agent gezielt Dateiansicht, Codesuche und Entitätssuche nutzen, wenn der erste Kontext nicht genügt.	Werkzeuge liefern weiteres Material; Parser und Graph bestimmen weiterhin die technische Struktur.
Quellen nennen	Verknüpft die Antwort mit passenden Dateien, Zeilen oder Quellentiteln.	Das Backend akzeptiert nur Zitate, die aus Retrieval oder tatsächlich genutzten Werkzeugen stammen.

RAG LIEFERT Belegtes Material Ausschnitte, Herkunft und Zitieranker — nach Zugriff und Relevanz gefiltert.	LLM LEISTET Verstehen & vermitteln Es verdichtet, erklärt und macht Schlussfolgerungen samt Unsicherheit transparent.	SYSTEM SICHERT Nachvollziehbarkeit Parser, Graph und Quellenprüfung halten Struktur und Belege prüfbar.
--	---	---

Was das Sprachmodell wirklich beiträgt.

Das Sprachmodell bekommt nicht die gesamte Datenbank, sondern das von RAG vorbereitete Material. Es kann daraus eine verständliche Antwort bauen: Zusammenhänge erklären, Informationen vergleichen und begründete Schlüsse ziehen.

Es darf dabei nicht so tun, als wären Vermutungen sichere Fakten. Quellen bleiben prüfbar, und wenn die Belege nicht reichen, sollte die Antwort das deutlich machen. Der Parser und der Graph liefern weiterhin die feste technische Struktur.

Merksatz: Das LLM vermittelt und begründet; RAG recherchiert und begrenzt die Belege.

■ CITATIONS & SOURCE VALIDATION

Eine generierte Antwort wird erst nach Quellenauflösung zur nachvollziehbaren Arbeitsantwort.

Candidate sources

Vektortreffer und tatsächlich genutzte Agent-Werkzeuge liefern eine Kandidatenmenge.



Answer cites

Die Antwort nennt Datei-/Zeilenreferenzen oder bekannte Quellentitel nach vereinbarten Formaten.



Backend validates

Nur zitierte Kandidaten, die wirklich im Retrieval/Tooling vorkamen, werden als Quellen akzeptiert.



UI links back

Referenzen können wieder zu Datei, Zeile, Dokumentseite oder Kontextansicht führen.

Warum diese Zusatzschicht? Ein Modell kann plausible, aber nicht existierende Pfade nennen. Die Quellenauflösung koppelt sichtbare Belege daher an den tatsächlichen Researchweg statt allein an generierten Text.

■ EINFACH ERKLÄRT • THEMA 20

Quellen machen Antworten überprüfbar.

Eine gut klingende Antwort ist nicht genug, wenn niemand sehen kann, worauf sie beruht. Doctus verbindet Antworten deshalb mit Dateien, Zeilen oder Dokumentseiten, die bei der Recherche tatsächlich verwendet wurden.

Das Backend prüft diese Verweise. Ein Sprachmodell kann also nicht einfach einen plausibel klingenden Dateinamen erfinden und ihn als Beleg ausgeben. In der Oberfläche führen die Quellen wieder zurück zum Original.

Merksatz: Ein Beleg ist nur dann wertvoll, wenn er zum tatsächlichen Researchweg zurückführt.

Das Frontend ist ein konfigurierbarer Arbeitsraum, kein einzelnes Chatfenster.

Next.js orchestriert eine React-Anwendung mit mehreren gleichzeitig offenen Ansichten. Der zentrale Seitenzustand verteilt Panelkonfiguration, Auswahl, Fokus, Historie und Workspace-Snapshots an spezialisierte Komponenten.

SHELL	CODE	GRAPH	SETTINGS
app/page.tsx Orchestriert Projekt, Panels, Selektionen, Chat- und Workspace-Zustände.	Monaco Editor Zeigt Quelldateien; Klicks und Marker können aus einer Zeile einen referenzierbaren Fokus machen.	Graph views Call Graph und Knowledge Graph visualisieren gefilterte Knoten/Kanten aus API-Ergebnissen.	Admin & Sources Quellen-, KI-, Team-, Nutzer- und Projektverwaltung folgen dem Berechtigungsmodell der API.

Designabsicht: 1–4 Panels erlauben, Code, Graph, Chat und Kontext nebeneinander zu halten. Fixierte Views und Panelhistorien verhindern, dass eine Navigation den Arbeitsstand ungewollt verliert.

Der Arbeitsbereich ist mehr als ein Chat.

Beim Verstehen eines alten Systems möchte man oft gleichzeitig Code lesen, eine Beziehung ansehen und eine Frage stellen. Doctus erlaubt deshalb mehrere Ansichten nebeneinander, statt alles in ein einziges Chatfenster zu zwingen.

Die Anwendung merkt sich, was gerade ausgewählt oder geöffnet ist. So kann man einem Hinweis folgen, ohne den bisherigen Arbeitsstand zu verlieren. Der Chat ist dabei ein Teil des Werkzeugs, nicht sein einziger Zugang.

Merksatz: Mehrere Ansichten halten Untersuchung, Kontext und Gespräch zusammen.

Fokusobjekte verbinden UI-Aktionen mit technischen Datenankern.



UI-BAUSTEIN	TECHNISCHE FUNKTION
Code View	Monaco-Dekorationen und Focus-Navigation verbinden Entity-/Zeileninformationen mit dem sichtbaren Originalcode.
References / Call Graph	API-Abfragen liefern gruppierte Inbound-/Outbound-Kanten; Filter und Hop-Anzahl steuern die Projektion.
Chat View	Persistiert Nachrichten, Metadaten, Referenzen und optional Workspace-Snapshots pro Sitzung.
Job Center	Visualisiert den Status von Hintergrundverarbeitung und schafft eine UI-Brücke zwischen Celery und Nutzer.

■ EINFACH ERKLÄRT • THEMA 22

Ein gemeinsamer Fokus verbindet die Ansichten.

Wenn jemand in einer Ansicht auf eine Codezeile, ein Dokument oder einen Graphknoten klickt, beschreibt ein gemeinsamer Fokus genau dieses Objekt. Andere Ansichten können darauf reagieren und den passenden Ausschnitt zeigen.

So wird aus einem Klick kein Sprung ins Unbekannte: Eine Zeile kann im Chat als Referenz erscheinen, ein Graphknoten kann die passende Datei öffnen und die Navigation bleibt nachvollziehbar.

Merksatz: Der Fokus ist der gemeinsame Zeigefinger zwischen allen Bereichen der Oberfläche.

DEVELOPER MAP

Die wichtigsten Einstiegspunkte nach Entwicklungsaufgabe.

WENN DU ÄNDERN WILLST ...	BEGINNE HIER	TYPISCHE NACHBARN
COBOL-Syntax / Entitäten / Kanten	parser/cobol/parse.py	source_format.py, divisions.py, data_division.py, procedure.py, xref.py, Golden-Tests.
Persistenz und Nachauflösung	parser/cobol_persist.py	parser/tasks/edge_resolver.py, ORM-Modelle, Migrationen.
Git-/Dokument-Ingestion	parser/connectors/git.py bzw. parser/connectors/	chunk_reindex.py, ollama_client.py, Source-Modelle.
RAG / Chat / Quellen	backend/api/chat.py	services/graph_retrieval.py, Agent-/MCP-Code, Projekte/Teams.
Login / Rollen / Projekte	backend/api/auth.py, core/teams.py, core/projects.py	auth_dependency.py, api/teams.py, api/projects.py.
Arbeitsraum / Panel-UX	frontend/app/page.tsx	frontend/components/, app/services/api.ts, Views und Settings-Tabs.

■ EINFACH ERKLÄRT • THEMA 23

Wo Änderungen am besten beginnen.

Ein großes Projekt wirkt leichter, wenn man die richtige Einstiegstür kennt. Die Entwicklerkarte zeigt deshalb für jede typische Änderung den passenden Bereich im Repository – etwa für Parser, Quellen, Chat, Rechte oder Oberfläche.

Daneben stehen die wichtigsten Nachbarn. Das spart Suchzeit und macht sichtbar, welche Teile einer Änderung wahrscheinlich ebenfalls betroffen sind. Wer etwas anfasst, kann so gezielter testen.

Merksatz: Die Karte ersetzt nicht das Lesen des Codes, aber sie verkürzt den ersten Weg dorthin.

 SUMMARY

Doctus verbindet deterministische Struktur mit semantischer Assistenz.

CODE VERSTEHEN**Parser + Graph**

COBOL wird in stabil referenzierbare Entitäten und gerichtete Abhängigkeiten übersetzt.

WISSEN FINDEN**RAG + Quellen**

Vektor- und Referenzsuche werden mit strukturellem Kontext und validierten Quellen kombiniert.

SICHER ARBEITEN**Local + scoped**

Lokale Datenhaltung, verschlüsselte Inhalte sowie Team-/Projektzugriff formen den Arbeitskontext.



Doctus.

Kein Chatbot mit Code-Anhang, sondern ein Arbeitsraum, in dem Originalquellen, strukturelle Beziehungen und semantischer Kontext zusammenspielen.

Weiterführende Repository-Dokumente: [README.md](#), [docs/IMPLEMENTIERUNGSPPLAN.md](#), [docs/ENTSCHEIDUNGEN.md](#), [docs/DEPLOYMENT.md](#) und [docs/produkt-handbuch-doctus.html](#).

Das Gesamtbild in einem Satz.

Doctus verbindet originale Unterlagen mit einer festen technischen Struktur und einer Suche nach Bedeutung. Daraus entsteht ein Arbeitsraum, in dem Teams komplexe Altbestände schrittweise verstehen können.

Wichtig ist die Aufgabenteilung: Regeln und Parser liefern verlässliche Fakten, RAG stellt passende Belege zusammen und das Sprachmodell hilft beim Erklären. Alles bleibt an Quellen, Rechte und nachvollziehbare Schritte gebunden.

Merksatz: Doctus macht Wissen zugänglich, ohne seine Herkunft oder seine Grenzen zu verstecken.